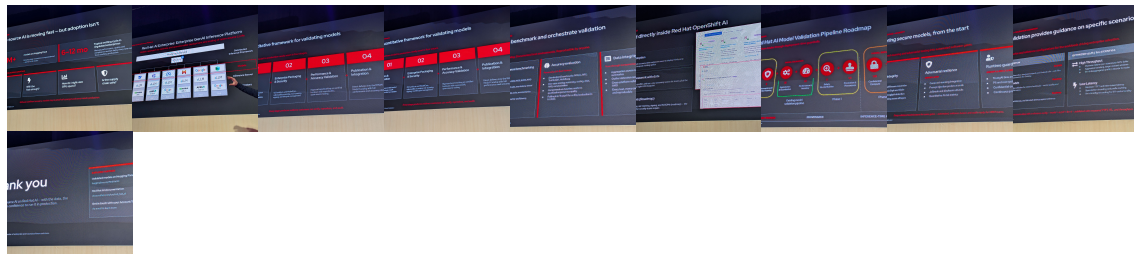


## chain\_of\_trust

 Tue, May 12, 2026 1:34PM  20:24

## SUMMARY KEYWORDS

Open source AI, model adoption, vetting cycle, quality metrics, GPU compatibility, supply chain safety, Red Hat AI, model lifecycle, performance benchmarking, model quantization, security validation, adversarial auditing, model provenance, confidential compute, runtime guardrails.



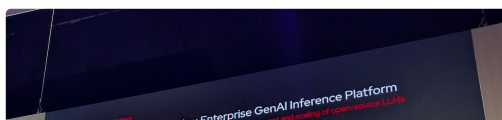
## SPEAKERS

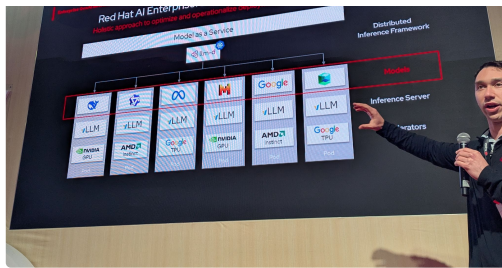
Speaker 1, Speaker 3, Speaker 4, Speaker 2

S

## Speaker 1 00:00

Oh, open source AI is moving extremely fast, right? But adoption is not right. Some of the reasons why that is there are over 2.4 million models on hugging right. Every single week, there are new models coming out different providers, whether it be Mistral or Nvidia or Gwenn and Alibaba, cloud or deep sea, the models continue to land on hugging face, but customers are really struggling to take these models and the latest innovations in AI and actually implement them within their businesses. Part of that reason is the vetting cycle that it can really take to when a model comes out. It's more than just evaluating the license itself. We're really looking at, you know, the processes that need to be done in these in a lot of these regulated industries, to ensure that model is safe, has the right quality metrics that we're looking for, has the right performance, right? As you think about all the different vectors of models that you need to evaluate, this can end up taking six to 12 months. So, hey, a model comes out, you can't just take that model and deploy it to production, right? You have to go through this process the cycle pretty sure you got the right way. And then a lot of the other questions that we get from customers that's blocking adoption are things like, will it run on my GPU, right? Will it be fast enough for my use case, right? And that will depend whether you're doing chat or RAD or coding, right. These will affect whether your model is fast enough. How do I right size my GPU spend? Right? How do I not over allocate or under allocate? And then lastly is the supply chain safety, right? You can't just go and really willy nilly download models off the internet without knowing what's in there. So that is kind of a new vector that we're also really focused on. Also quick poll by show of hands, who here works at a company that doesn't have access to hugging face on their network, put your hand up. You can't get onto hugging face at all. Awesome. Yeah, these are a lot of the core issues we're looking to solve. So at Red Hat AI, right? We have a full enterprise staff, starting all the way at the top with how you serve models, getting into llm, and vllm, which is the core engine. That is how you serve those models effectively. And then, of course, a fully hardware agnostic layer for the accelerator piece. Today we're really focusing on the model piece, right? So you can see here there's a well known amount of players and a lot of these other areas right on the accelerator side, we have, you know, all the usual suspects, bllm in the middle. And then, of course, the models which we just spoke about their 2.4 million plus on hugging face. How do you start thinking about

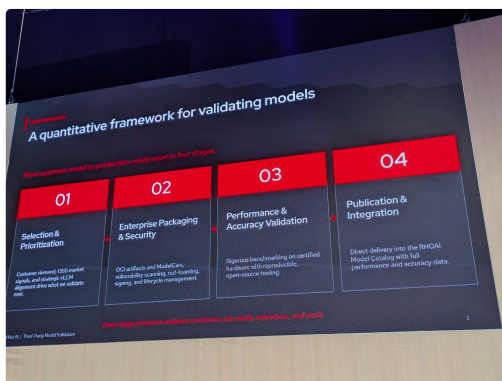


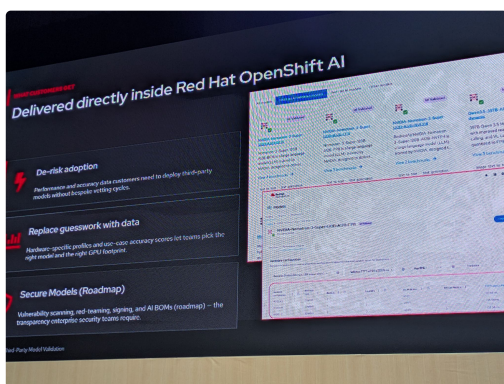
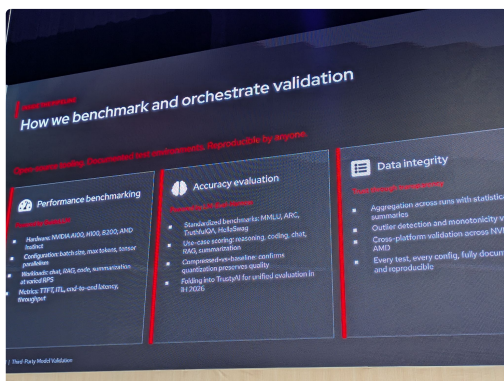


S

## Speaker 2 02:31

this lifecycle? So we start off by selecting by activities as well as the model to our service, we package all these models in Red Hats, packing formats, OCI artifacts, negative safety and security validation. It, and importantly, that means both to work themselves as well as the behavior. Secondly, thirdly, we then do progress and accused validation, so we actually deploy the models on top of our slides of accuracy and progress metrics finally, we take all of the models and the data about them and publish them in AI. Importantly, all part of the conversation itself. So we are taking everything that we are gathering about models and streaming deeper about how the validation pipeline actually works. So the first thing we can do is here, we're using gardener lab. I've been wearing that on top of certified workers, day to day, gathering about these things at the same time with the accuracy validation. Accuracy validation using tests are to standard use cases. Of data before we show that to you guys, we are getting all this together for outliers when We see all of the messages that we gathered about gives to us is the ability to try to do the same amount of custom validation we're able to rather than guessing precisely what the hardware we need, and use our validated results to improve that through each way that was al bonds, red teaming. So we talked





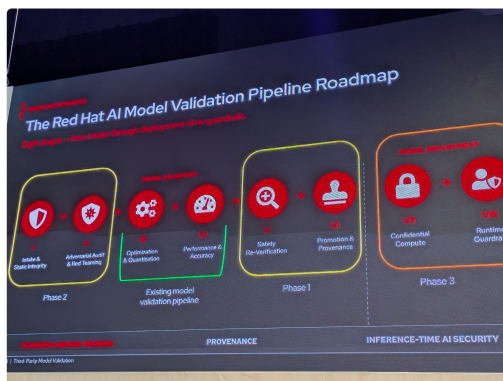
S

## Speaker 1 06:00

about kind of our current validation pipeline, right, where we do performance benchmarking, evaluate the models. The other piece of the evaluation is we often progress these models, so we have a red hat AI hooking face the public as well through our model catalog that we just showed here. Really the goal of doing that evaluation is to show that when we quantize models and compress them, we're still able to maintain within 95 plus percent of that baseline accuracy. So you can take a model, cut it in half in terms of size and memory allocation that's needed, and then basically you can run that model get up to two plus x performance while sacrificing very minimal problems. So in terms of like the roadmap, right? So right now, we're very much in stages three and four. So we started in the middle, but really it's around optimization and quantization, which is that step that we do to make sure that we're offering compressed models that can improve performance, right? And performance will that drive efficiency, which will enforce the job cost and the performance and accuracy piece, right? So we optimize the model, the benchmark it we spin the model up on the yellow line. We send simulated workloads, right? We scale into our users, different workload shapes, different prompts sizes, going in to simulate these different workloads. And then, of course, run the accuracy to kind of prove that we haven't actually tanked the quality where we're planning to expand next is really thinking about security, right? So AI has been moving really, really fast. Some, some might argue too fast, you know, I'm sure everyone has saw, seen the news around with those and all the vulnerabilities that have been uncovered, both within models and just software in general. So thinking about that, a lot of our customers will come to us and say, you

know, have said, how do we think about deploying secure models? Right? 68% of models last year put on hugging face came out of China's lab and Chinese companies. So as you consider, you know, some of the biases that exist with those models and some of the risks, right? For example, toxicity, right? If you ask a model to do something that's toxic, or you ask a model to like violence, right, you can ask a model, how do I build a bond you work on a bank. You don't want a quen model telling your IT engineer how to build right? That doesn't make a lot of sense. So what we're planning to zero for these models to extend out so on the bus side, starting from phase one, doing intent and static integrity testing, right? So this is really think about treating and model as code, right? All the model is series of weights, right, tensors. And you can actually do scans right, like static analysis, scans on those models to ensure that when you take that, when Red Hat. Takes that model for hugging face. It's put out there by the original provider, and we send it through our packaging pipeline. We aren't packaging. There's a python file that's there one. Why is it there? What's in that file? Right? Does it have anything malicious in it? These are the scans that you kind of need to be thinking about when you just go and grab the latest open source products. The second piece here is around red teaming, adversarial auditing, so I talked a little bit just a second ago about toxicity, about violence, leaking PII, right, leaking confidential information that should be kept in house. How do you make sure models aren't susceptible to things like violence? Prompt, injection, jailbreak, right? If you ask a model to leak all of the API keys that it may or may not have access to in English, it might not do that. But if you go to Google Translate and you translate that to a different language that the model might be able to understand, it might go ahead and do it, right? So these are kind of different attack vectors that you can think about, that we're trying to start testing for all of the models within the model catalog. So we will use Derek, which is an NVIDIA open source project that sends probes at these models, and basically, we'll evaluate them across all these different attack spectrums, jailbreaking, violence, sexual content, these types of things, to ensure that when you roll this out from the enterprise, you're not kind of leaving the door open for any of these kind of attacks. Then we'll go through our normal optimization and performance and accuracy pieces, and on the right side, we get into model provenance, right? So how do you actually track that? When you take a model from hugging face, we package it up as Red Hat through our supply chain, it ends up in the catalog, and if your team gets their hands on it, how do you actually track the lineage of that model? Right? And we do that through model signing and model verification. So if you have different different engineers that are going in taking the model, maybe they're doing more compression on it, maybe they're fine tuning it, right, and say, Hey, let's align this to our data. You basically every single step in that process will be have a signature, and then you basically have a ledger that can't be tampered with to understand the life cycle of that model. Right? This helps make sure that you don't have people taking models in the model that Red Hat packages and has gone through that whole left side of security scanning and Red Teaming is the actual one you're getting right. You don't have a bad actor going in, uploading their own model from GitHub or wherever else, into your cluster and deploying those models. That's on the model validation packaging side three. On the right side here is all around inference time, AI security, right? So everything on the left is all around models, right? These are before the models are deployed in production. This is a red hat curating a list of models that have data results you can reproduce yourself and scan all within catalog, and that enables you to make that decision of, okay, this model looks safe, looks performance, it looks like it's high quality. I'm willing to move forward. On the right side, we get into model deployment. So these are two new kind of areas of technology that we're investing in, the first one being confidential containers and confidential compute, right? So you have a model that's deployed. How do you basically think about making the container and the way that you're deploying it into a total black box, right? So the model get all the all the boxes is doing is taking prompts in and spinning out generation tokens, but there is actually no way to understand to see the weights of that model, or potentially reverse engineering was

this is a way to kind of serve models in a more protected way, so that you can't get access to any details around that model itself. All it is giant black box that is taking input and generating output. And the last one step eight here is thinking about using guardrails. So I don't know how many of you are familiar with that. Trust the AI, but our Nemo guardrails as well. But basically, when you think about deploying models like I talked about before, if you ask Gwen how to build a bomb, the older Gwen models will tell you, right, we don't want that in our enterprise. So before the model even gets that request, how can we set up guardrails around our deployment to make sure that that request never even gets through? Right? If you're asking a totally irrelevant, dangerous, toxic question, why you can spend the tokens to see what the output is if it does it or not, and that's where runtime guardrails can really come into play as think about it as a proxy layer between your user, your client, the actual model running on the LLM cluster and going back to the client. And you can set up guardrails both at the prompt stage. So when I ask it, you know, how do I want to leak all my customer information? Set this list out as a attack and weakness on predator, right? It won't even let that go to the model. Say, No, you're asking to do something that is against our policy, right? You can provide rules. You can use our box guardrails to kind of catch this stuff. The same thing goes on the output side, right? If you were to get a prop through and the model starts generating, you know, to build the ball first fly by this chemical, right? It'll stop itself before that generation goes out and say, No, this is against our guardrail rules. Let's not give this back to the user, right? Starting to generate toxic information. So these are kind of the new vectors. Sorry, you have a question. So, yeah, so the phases are left broad for a reason. Phase one is something we're already working on on the engineering side. Phase two is also something we're working on. So I'm hoping phase one and two come by the end of the summer. Again, that's a my my guess is not promise, but definitely by the end of this year, we're planning to integrate security into our dollar packaging pipe, then on the right side of confidential containers. This is more of a research effort right now that we're actually doing with Nvidia. So Nvidia has their own confidential computing projects that we're collaborating with them on, as Red



S

Speaker 3 14:36

Hat, that's probably a little

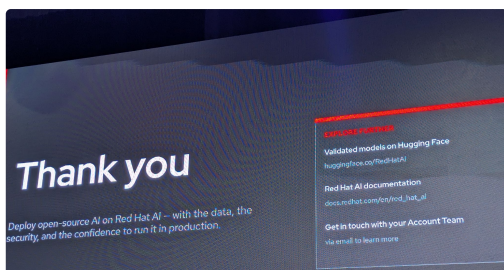
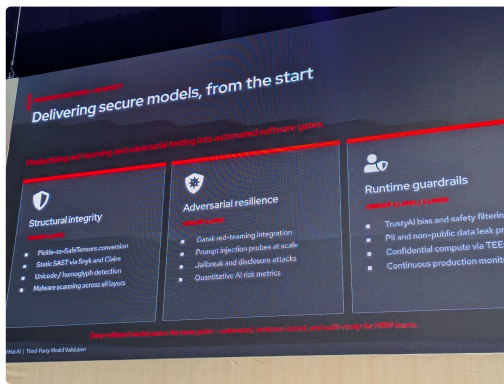
S

Speaker 1 14:39

bits of longer timeline in guardrails. We actually have guardrails GA in the product already. So if

bit or longer timeline in guardrails. we actually have guardrails GA in the product already. So if you go to the guardrail section under trust the AI, which is the open source project, and I'll curate the guardrails, that's already GA, so it's in phase three. Model Deployment side, you can actually deploy models with guardrails and OpenShift. AI, okay, little more high level, but this will kind of bring it home areas, delivering secure models from the start, right? This is something that you can't get on hugging face you can't get on GitHub. So starting with structural integrity, right? We talked about treating models as code, doing scans like static sasd received Claire, are kind of some examples, malware scanning, looking for cdds, understanding what is actually in the models themselves that might be at risk. Adversarial resilience gets into Eric gray in that library, setting these probes at the model and seeing how do you break this thing right? Is this model really bad when I start asking it to leak customer information. So, hey, there's a bunch of emails. Send these out, and it actually doesn't right. That's something we would want to know. And how to mitigate those hardware rails on the right side, you have to get into Barbara Trust, the AI, getting into safety filtering, not leaking confidential information, and then obviously getting into confidential compute, making sure that you're implementing the same equipment is safe and continuous monitoring right. Obviously, you ought to still do the standard machine learning data science, things like making sure the model isn't drifting. Outputs aren't changing a lot, because they can be susceptible to getting prompts in and adjusting their behavior right, especially when you have multiple users hitting that safety so like we talked about model validation on the performance side and evaluation side, we target specific scenarios that Tom spoke about chat rag summarization. We're also planning to extend the coverage in terms of scenarios here, so new scenarios that are evolving projected workflows like tool calling was a really big one that would be able to laugh, right? So how do we go in and validate that a gem of four or a Nemo Tron 120 B model will actually be effective at the tool calling in on the OpenShift AI product, right? And that's kind of the process of our team going out running benchmarks like virtual calls before running benchmarks like SW benchmark verified and actually going in and building the demos within our products, like taking demo four and plugging that into Cloud code and saying, build me a Python application that can generate images, and just letting it go and watching an open source model power Cloud code is one way that we like test does this tool going actually work right? You don't need opens. You don't need CPT five. When you have models like when, like when, 3.6 general, for Nemo Shawn, they're actually really giving the model. So that starts at the BLM level. It goes all the way up through the bottle catalog. So hopefully by the end of this quarter, we'll see green stamps in the catalog of this model validated with tool calling. So for any agentic workflows, any rag applications we want to leverage that capability, they'll be very easily discovered. Another use case, embedding models, right? So how do you make sure that embedding models are effective? We can actually go and measure the quality of that and how that affects rag pipelines as well, right? Like, how do I make sure that when I set when I set this stuff up and I set my team down rabbit hole spending hours, we're going to be successful, right? And because there are 2.4 million models out there, Red Hat AI has over 700 quantized models in our own hugging face subset of that. But we also shipped in the catalog to go through this. But how do you make sure that these models are going to be affected what you're trying to do, right? You're not just spinning your wheels, and that's really where we're trying to take model validation, right? These specific use cases, we want to make sure that we're setting you up for success. So you don't install models that are too large for your team members, so you don't run a model that has very, very terrible tool calling and you go, Oh, this isn't really working in crypto, right? How do we avoid that pain? And that's kind of what we're targeting. To address the other piece here on the right side. So there's one certainly to be said about the actual use case, but tying it back to performance, right? Different use cases have different metrics. You should be optimizing BLM or II before, right? If you want to do a chat bot, you're going to want to think about a low latency use case, right? You want to fast time your first token. You don't want to send a request

to a bot and delay. No one wants to do that, right? So if you're doing a chat use case, we can actually provide the optimal vllm configuration arguments, right within the model, right? And all you do is click a button and say, I want to deploy this model with low latency in mind on my GPUs, and it'll automatically populate the right arguments. Right? Should do service with tensor parallelism. Should do service with streaming on or off. Right? These decisions that a BLM expert that is up to date on the talks would know, but you know, as people working in these companies, when the space is moving so fast, we want to make that a lot easier, right? Same goes for high throughput scenarios, right? You want to do offline matching. You want to take large sums of documents, summarize them. You don't care about how fast that time. You don't care if it's snapping now you can run that model with a high throughput scenario and really take care and really optimize your through code utilization. You okay, that's all I had. Yeah, we're basically at time. I could take two questions. I think. Thank you all for joining again. I'm Rob. Feel free to reach out and find me after





Speaker 4 20:20  
here's the.